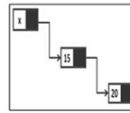


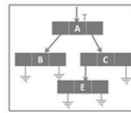
Veri Yapıları



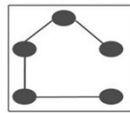
Sorting



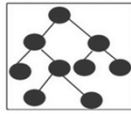
Link list



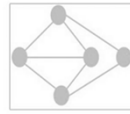
list



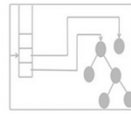
spanning tree



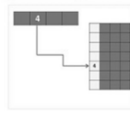
Tree



Graph



Stack



Hashing

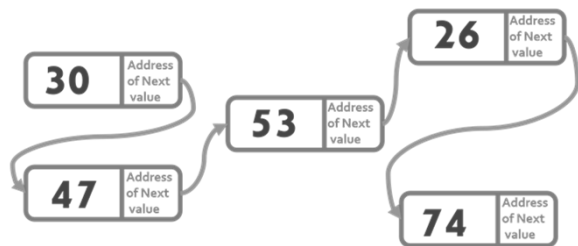
By ... sarikkamanchiphotography

Hafta 8

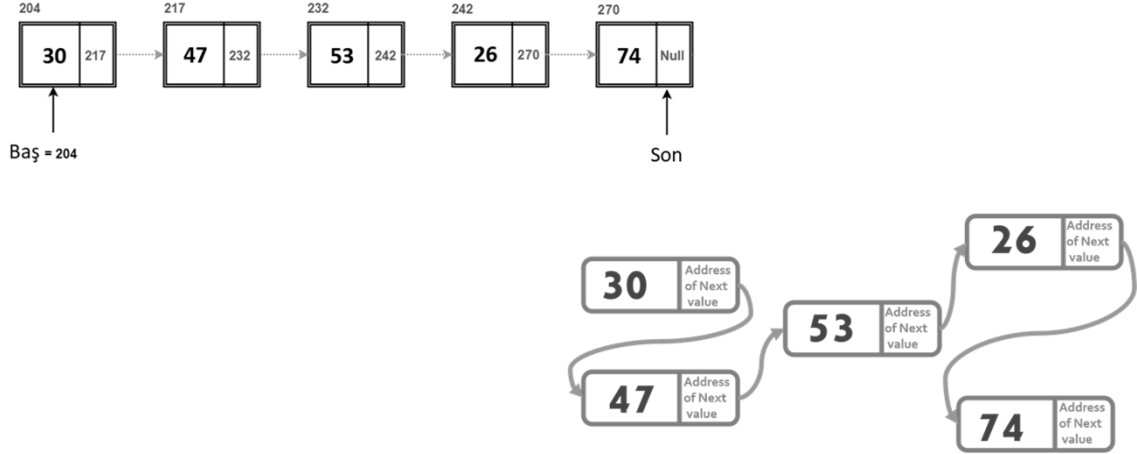
BAĞLI LİSTELER (Linked Lists)

- Yığın ve Kuyruk veri yapılarında elemanlar hafızaya ardışık bir şekilde yerleştirilmekteydi.
- **Bağlı listelerde elemanlar hafızaya ardışık olarak yerleştirilmez.**
- Her eleman kendisinden sonra gelen elemanın adresini de tutar.
- Bir dizinin içindeki elemanlara dizinin indisleri yoluyla ulaşılırken, bir bağlı listenin içindeki elemanlara o elemanlara işaret eden işaretçiler yoluyla ulaşılır.

- Eleman → Düğüm (Node) = veri + adres
- Adres → sonraki düğümün adresini tutan işaretçi



BAĞLI LİSTELER (Linked Lists)



Düğüm Sınıfı

```
class Dugum
{
public:
    Dugum()
    {
        pSonraki = NULL;
    }
    int Veri;
    Dugum* pSonraki;
};
```

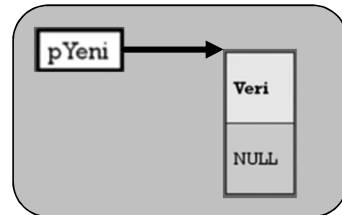
İşaretçi bir yer göstermeyecek

Çünkü bu işaretçi bir başka düğümün adresini tutacak.

Yeni bir düğüm oluşturmak istediğimizde:

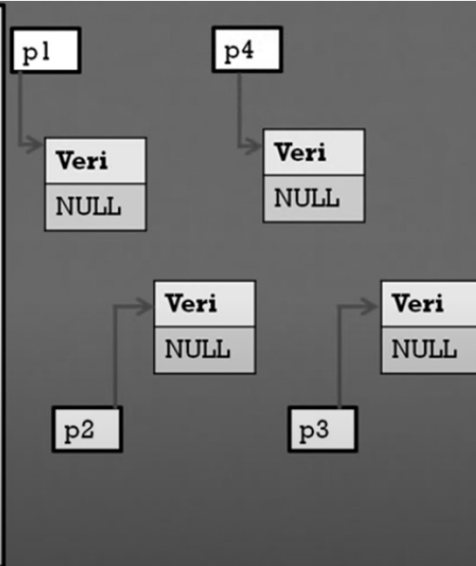
```
Dugum* pYeni = new Dugum();
```

Böylece delete kullanılmadığı sürece düğümü kaybetmeyiz



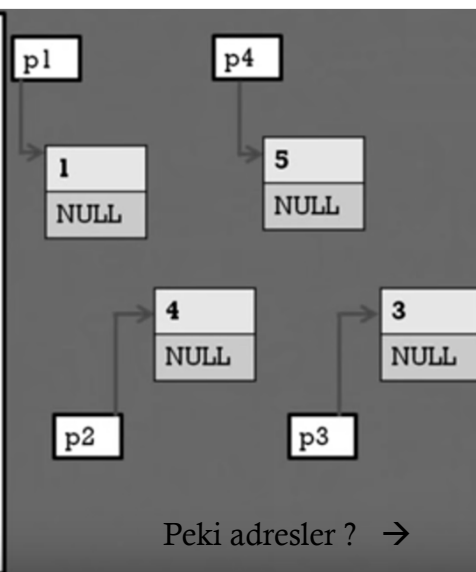
BAĞLI LİSTELER

```
int main()
{
    Dugum* p1 = new Dugum();
    Dugum* p2 = new Dugum();
    Dugum* p3 = new Dugum();
    Dugum* p4 = new Dugum();
}
```



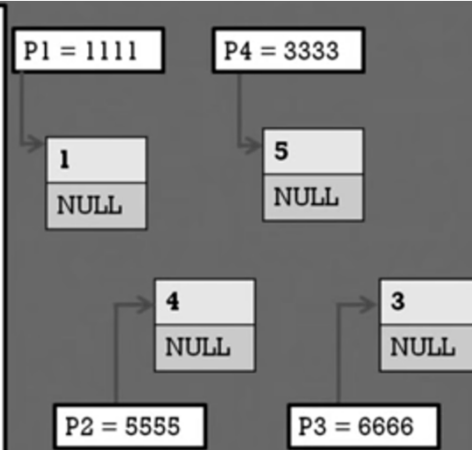
BAĞLI LİSTELER

```
int main()
{
    Dugum* p1 = new Dugum();
    Dugum* p2 = new Dugum();
    Dugum* p3 = new Dugum();
    Dugum* p4 = new Dugum();
    p1->Veri = 1;
    p2->Veri = 4;
    p3->Veri = 3;
    p4->Veri = 5;
}
```



BAĞLI LİSTELER

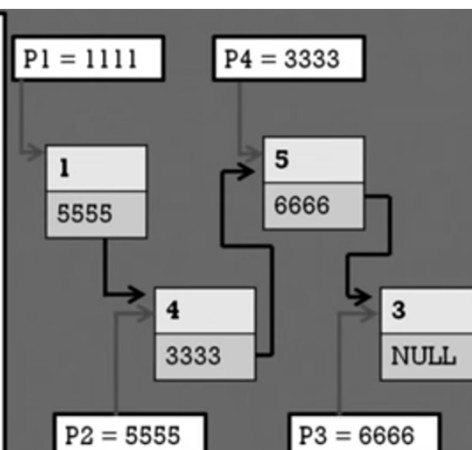
```
int main()
{
    Dugum* p1= new Dugum();
    Dugum* p2= new Dugum();
    Dugum* p3= new Dugum();
    Dugum* p4= new Dugum();
    p1->Veri= 1;
    p2->Veri= 4;
    p3->Veri= 3;
    p4->Veri=5;
}
```



Peki düğümler nasıl bağlanacak ? →

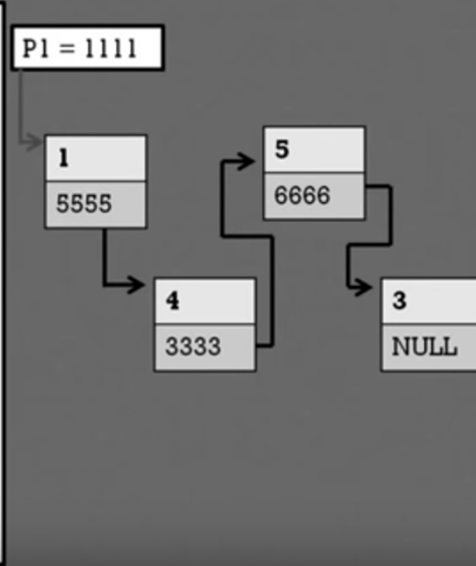
BAĞLI LİSTELER

```
int main()
{
    Dugum* p1= new Dugum();
    Dugum* p2= new Dugum();
    Dugum* p3= new Dugum();
    Dugum* p4= new Dugum();
    p1->Veri= 1;
    p2->Veri= 4;
    p3->Veri= 3;
    p4->Veri=5;
    p1->pSonraki= p2;
    p2->pSonraki= p4;
    p4->pSonraki= p3;
}
```



Listenin ilk düğümünün adresi ÖNEMLİ

```
int main()
{
    Dugum* p1 = new Dugum();
    Dugum* p2 = new Dugum();
    Dugum* p3 = new Dugum();
    Dugum* p4 = new Dugum();
    p1->Veri = 1;
    p2->Veri = 4;
    p3->Veri = 3;
    p4->Veri = 5;
    p1->pSonraki = p2;
    p2->pSonraki = p4;
    p4->pSonraki = p3;
}
```



BAĞLI LİSTEYİ YAZDIRALIM

```
int main()
{
    .....
    .....
    Dugum* pTemp = p1;

    while(pTemp != NULL)
    {
        cout << pTemp->Veri;
        pTemp = pTemp->pSonraki;
    }
}
```

Adres tutan
geçici bir işaretçi

